Lab 03 · Pipeline completa con AWS CodePipeline

Duración estimada: 60 minutos **Sesión:** 2 **Entorno:** AWS CloudShell y consola · región `eu-west-1` (Irlanda)

Objetivo

Construir una pipeline de entrega continua con cinco etapas: **Source** (zip en S3) → **Build** (pruebas y empaquetado con CodeBuild) → **Deploy-Pre** (CloudFormation) → **Aprobación** manual → **Deploy-Pro** (CloudFormation con el mismo artefacto). Después publicarás una nueva versión y la seguirás de principio a fin.

Requisitos

- Permisos sobre CloudFormation, IAM, S3, CodeBuild, CodePipeline, Lambda y EventBridge en la cuenta de formación.

Punto de partida

La práctica es independiente. En el paso 1 despliegas un stack que crea el bucket de origen y artefactos, el proyecto de CodeBuild y los roles necesarios.

> **Nota de seguridad.** Para no consumir el tiempo de la práctica en ajustar permisos, el rol que usa CloudFormation para desplegar tiene permisos de administrador. **No reproduzcas esto en producción**: allí ese rol debe limitarse a los servicios y recursos que despliega la pipeline.

Paso 1 · Desplegar el punto de partida

```
export SUFIJO=jlb01            # tu sufijo, en minúsculas
export AWS_REGION=eu-west-1
mkdir -p ~/lab03 && cd ~/lab03

cat > inicio-lab03.yaml <<'EOF'
AWSTemplateFormatVersion: "2010-09-09"
Description: Lab 03 - punto de partida de la pipeline
Parameters:
  Sufijo:
    Type: String
Resources:
  Bucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: !Sub "lab03-${Sufijo}-${AWS::AccountId}"
      VersioningConfiguration: { Status: Enabled }
      NotificationConfiguration:
        EventBridgeConfiguration: { EventBridgeEnabled: true }

  RolCodeBuild:
    Type: AWS::IAM::Role
    Properties:
      RoleName: !Sub "lab03-codebuild-${Sufijo}"
      AssumeRolePolicyDocument:
        Version: "2012-10-17"
        Statement:
          - { Effect: Allow, Principal: { Service: codebuild.amazonaws.com }, Action: sts:AssumeRole }
      Policies:
        - PolicyName: permisos-build
          PolicyDocument:
            Version: "2012-10-17"
            Statement:
              - Effect: Allow
                Action: [logs:CreateLogGroup, logs:CreateLogStream, logs:PutLogEvents]
                Resource: "*"
              - Effect: Allow
                Action: [s3:GetObject, s3:GetObjectVersion, s3:PutObject, s3:GetBucketLocation,
```

```
s3:ListBucket]
                Resource:
                  - !GetAtt Bucket.Arn
                  - !Sub "${Bucket.Arn}/*"
                  - "arn:aws:s3:::codepipeline-*"
                  - "arn:aws:s3:::codepipeline-*/*"
              - Effect: Allow
                Action: [codebuild:CreateReportGroup, codebuild:CreateReport, codebuild:UpdateReport,
codebuild:BatchPutTestCases]
                Resource: !Sub "arn:aws:codebuild:${AWS::Region}:${AWS::AccountId}:report-group/*"

  ProyectoBuild:
    Type: AWS::CodeBuild::Project
    Properties:
      Name: !Sub "lab03-build-${Sufijo}"
      ServiceRole: !GetAtt RolCodeBuild.Arn
      Source: { Type: CODEPIPELINE }
      Artifacts: { Type: CODEPIPELINE }
      Environment:
        Type: LINUX_CONTAINER
        ComputeType: BUILD_GENERAL1_SMALL
        Image: aws/codebuild/amazonlinux-x86_64-standard:5.0
        EnvironmentVariables:
          - { Name: ARTIFACT_BUCKET, Value: !Ref Bucket }

  RolDespliegue:
    Type: AWS::IAM::Role
    Properties:
      RoleName: !Sub "lab03-cfn-${Sufijo}"
      AssumeRolePolicyDocument:
        Version: "2012-10-17"
        Statement:
          - { Effect: Allow, Principal: { Service: cloudformation.amazonaws.com }, Action: sts:AssumeRole }
      ManagedPolicyArns:
        - arn:aws:iam::aws:policy/AdministratorAccess   # SOLO FORMACIÓN

Outputs:
  Bucket: { Value: !Ref Bucket }
  RolDespliegue: { Value: !GetAtt RolDespliegue.Arn }
EOF

aws cloudformation deploy --stack-name lab03-inicio-$SUFIJO \
  --template-file inicio-lab03.yaml --parameter-overrides Sufijo=$SUFIJO \
  --capabilities CAPABILITY_NAMED_IAM

export BUCKET=$(aws cloudformation describe-stacks --stack-name lab03-inicio-$SUFIJO \
  --query "Stacks[0].Outputs[?OutputKey=='Bucket'].OutputValue" --output text)
echo "Bucket: $BUCKET"
```

Paso 2 · Aplicación, plantilla SAM y buildspec

```
mkdir -p app/src app/tests && cd app

cat > src/app.py <<'EOF'
import json
import os

VERSION = "1.0.0"


def saludo(nombre: str) -> str:
    return f"Hola, {nombre}"


def handler(event, context):
    nombre = (event or {}).get("nombre", "mundo")
    return {
        "statusCode": 200,
        "body": json.dumps({
            "mensaje": saludo(nombre),
            "version": VERSION,
            "entorno": os.environ.get("ENTORNO", "desconocido"),
        }),
    }
```

```
EOF

cat > tests/test_app.py <<'EOF'
import json
from src.app import saludo, handler


def test_saludo():
    assert saludo("Ana") == "Hola, Ana"


def test_handler_devuelve_version():
    r = handler({"nombre": "Luis"}, None)
    assert r["statusCode"] == 200
    assert "version" in json.loads(r["body"])
EOF
touch src/__init__.py tests/__init__.py

cat > template.yaml <<'EOF'
AWSTemplateFormatVersion: "2010-09-09"
Transform: AWS::Serverless-2016-10-31
Description: Lab 03 - aplicación desplegada por la pipeline
Parameters:
  Entorno:
    Type: String
    AllowedValues: [pre, pro]
Resources:
  Funcion:
    Type: AWS::Serverless::Function
    Properties:
      Runtime: python3.12
      Handler: app.handler
      CodeUri: src/
      Timeout: 5
      Environment:
        Variables:
          ENTORNO: !Ref Entorno
Outputs:
  NombreFuncion:
    Value: !Ref Funcion
EOF

cat > buildspec.yml <<'EOF'
version: 0.2
phases:
  install:
    runtime-versions:
      python: 3.12
    commands:
      - pip install pytest==8.3.3
  build:
    commands:
      - python -m pytest --junitxml=reports/junit.xml
      - aws cloudformation package --template-file template.yaml
          --s3-bucket $ARTIFACT_BUCKET --s3-prefix paquetes
          --output-template-file packaged.yaml
reports:
  pruebas-lab03:
    files: [junit.xml]
    base-directory: reports
artifacts:
  files: [packaged.yaml]
EOF

zip -r ../app.zip . -x "*.pyc" -x "__pycache__/*"
aws s3 cp ../app.zip s3://$BUCKET/fuente/app.zip
cd ..
```

> Fíjate en que la plantilla **no** fija el nombre de la función: CloudFormation lo genera a partir del nombre del stack. Así la misma plantilla sirve para `pre` y `pro` sin colisiones.

Paso 3 · Crear la pipeline: Source y Build

En la consola: **CodePipeline → Pipelines → Create pipeline**.

1. **Creation option:** *Build custom pipeline*.
2. **Pipeline settings:** nombre `lab03-${SUFIJO}` (con tu sufijo), *Execution mode*: **Superseded**, *Service role*: **New service role** (deja el nombre propuesto).
3. **Source:** *Source provider* **Amazon S3**, bucket `lab03-<sufijo>-<cuenta>`, *S3 object key* `fuente/app.zip`, *Change detection*: **Amazon CloudWatch Events / EventBridge (recommended)**.
4. **Build:** *Other build providers* → **AWS CodeBuild**, proyecto `lab03-build-<sufijo>`. Tipo de build: *Single build*.
5. **Test:** *Skip test stage*.
6. **Deploy:** *Deploy provider* **AWS CloudFormation**, con estos valores:

Campo	Valor
Action mode	Create or update a stack
Stack name	`lab03-app-pre-<sufijo>`
Artifact name / File name	`BuildArtifact` · `packaged.yaml`
Capabilities	`CAPABILITY_IAM`, `CAPABILITY_AUTO_EXPAND`
Role name	`lab03-cfn-<sufijo>`
Parameter overrides	`{"Entorno": "pre"}`

7. Revisa y pulsa **Create pipeline**. La primera ejecución arranca sola.

> **Punto de control 1.** Las tres etapas terminan en verde. En **CloudFormation** aparece el stack `lab03-app-pre-<sufijo>`. Pruébalo:

```
FN_PRE=$(aws cloudformation describe-stacks --stack-name lab03-app-pre-$SUFIJO \
  --query "Stacks[0].Outputs[?OutputKey=='NombreFuncion'].OutputValue" --output text)
aws lambda invoke --function-name $FN_PRE --payload '{"nombre":"Ana"}' \
  --cli-binary-format raw-in-base64-out /dev/stdout
```

Paso 4 · Añadir aprobación y producción

1. Abre la pipeline y pulsa **Edit**.
2. Debajo de la etapa de despliegue, **+ Add stage** → nombre `Aprobacion`. Dentro, **+ Add action group**: nombre `AprobarProduccion`, *Action provider* **Manual approval**. Opcionalmente escribe en *Comments* “Revisar la versión desplegada en pre antes de aprobar”.
3. **+ Add stage** → `Deploy-Pro`. Acción `DesplegarPro`, *Action provider* **AWS CloudFormation**, con los mismos valores que en el paso 3 salvo:
   - Stack name: `lab03-app-pro-<sufijo>`
   - Input artifacts: `BuildArtifact`
   - Parameter overrides: `{"Entorno": "pro"}`
4. Renombra la etapa de despliegue original a `Deploy-Pre` si quieres que se lea mejor (opcional).
5. **Save** y después **Release change**.
6. Cuando la ejecución llegue a `Aprobacion`, pulsa **Review**, escribe un comentario y **Approve**.

> **Punto de control 2.** Tienes dos stacks (`pre` y `pro`) desplegados desde **un único build**. Compruébalo en el historial de la ejecución: la etapa Build se ejecutó una sola vez.

Paso 5 · Publicar una nueva versión

```
cd app
sed -i 's/VERSION = "1.0.0"/VERSION = "1.1.0"/' src/app.py
sed -i 's/return f"Hola, {nombre}"/return f"Hola, {nombre}. Bienvenido al curso de DevOps"/' src/app.py
sed -i 's/assert saludo("Ana") == "Hola, Ana"/assert saludo("Ana").startswith("Hola, Ana")/'
tests/test_app.py
zip -r ../app.zip . -x "*.pyc" -x "__pycache__/*"
aws s3 cp ../app.zip s3://$BUCKET/fuente/app.zip
```

```
cd ..
```

La subida dispara la pipeline automáticamente mediante EventBridge. Síguela en la consola:

1. Cuando termine `Deploy-Pre`, invoca la función de `pre` y comprueba que devuelve la versión `1.1.0`.
2. Invoca la de `pro` **antes** de aprobar: debe seguir en `1.0.0`.
3. Aprueba y, al terminar, comprueba que `pro` ya devuelve `1.1.0`.

```
FN_PRO=$(aws cloudformation describe-stacks --stack-name lab03-app-pro-$SUFIJO \
  --query "Stacks[0].Outputs[?OutputKey=='NombreFuncion'].OutputValue" --output text)
aws lambda invoke --function-name $FN_PRO --payload '{}' \
  --cli-binary-format raw-in-base64-out /dev/stdout
```

> **Punto de control final.** Has recorrido el ciclo completo: un cambio en el código llega a producción con pruebas, despliegue en preproducción y aprobación explícita, sin ninguna acción manual fuera de la pipeline.

Reto opcional

Sube una versión con una prueba rota (por ejemplo, cambia `assert saludo("Ana").startswith("Hola, Ana")` por `== "Adiós"`). Observa en qué etapa se detiene la pipeline, que ninguno de los dos entornos cambia y cómo se ve el informe de pruebas enlazado desde la acción de build.

Preguntas de reflexión

1. ¿Qué pasaría con una ejecución que está esperando aprobación si llega un cambio nuevo, dado que usamos el modo *Superseded*?
2. ¿Dónde añadirías pruebas de integración contra el entorno `pre` antes de la aprobación?

Limpieza

```
aws codepipeline delete-pipeline --name lab03-$SUFIJO
aws cloudformation delete-stack --stack-name lab03-app-pro-$SUFIJO
aws cloudformation delete-stack --stack-name lab03-app-pre-$SUFIJO
aws cloudformation wait stack-delete-complete --stack-name lab03-app-pre-$SUFIJO
aws s3 rm s3://$BUCKET --recursive
aws s3api delete-objects --bucket $BUCKET --delete "$(aws s3api list-object-versions --bucket $BUCKET \
  --query '{Objects: Versions[].{Key:Key,VersionId:VersionId}}' --output json)" 2>/dev/null
aws s3api delete-objects --bucket $BUCKET --delete "$(aws s3api list-object-versions --bucket $BUCKET \
  --query '{Objects: DeleteMarkers[].{Key:Key,VersionId:VersionId}}' --output json)" 2>/dev/null
aws cloudformation delete-stack --stack-name lab03-inicio-$SUFIJO
```

Borra también, desde IAM, el rol de servicio que creó la consola para la pipeline (`AWSCodePipelineServiceRole-eu-west-1-lab03-<sufijo>`) y, si quieres, el bucket de artefactos `codepipeline-eu-west-1-...` si no lo usa ninguna otra pipeline de la cuenta.